

07

강

머신러닝

선형 함수 근사

통계·데이터과학과 박준희



✧ 학습목차

- 1 선형기저함수 모델
- 2 정규화된 최소자승법
- 3 베이지안선형 회귀



1

선형 기저 함수 모델



1

선형 기저 함수 모델

● 선형 기저 함수 모델(Linear Basis Function Models)

입력 x 와 출력 y 사이의 관계는 x 의 기저 함수(basis function)를 사용한 선형 결합으로 포착할 수 있다고 가정

핵심 아이디어:

y 를 단순한 선형 함수로 직접 모델링하는 대신(예: $y = w_0 + w_1x$), 다음과 같이 모델링 한다:

$$y(x) = \sum_{j=0}^{M-1} w_j \phi_j(x),$$

이 형태에서 기저 함수(basis function) $\phi_j(x)$ 의 선택을 통해 복잡한 관계를 모델링 할 수 있다.

1

선형 기저 함수 모델

● 선형 기저 함수 모델(Linear Basis Function Models)

$$y(x, \mathbf{w}) = \mathbf{w}^T \boldsymbol{\phi}(x).$$

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{M-1} \end{bmatrix}, \quad \boldsymbol{\phi}(x) = \begin{bmatrix} \phi_0(x) \\ \phi_1(x) \\ \vdots \\ \phi_{M-1}(x) \end{bmatrix}.$$

- 다항식 기저: $\phi_j(x) = x^j$.
- Radial Basis Functions (RBF): $\phi_j(x) = \exp\left(-\frac{(x-\mu_j)^2}{2\sigma^2}\right)$.
- 푸리에 기저: $\phi_j(x) = \sin(jx)$ 또는 $\cos(jx)$.

1 선형 기저 함수 모델

● 선형 기저 함수 모델(Linear Basis Function Models)

다항식 모델을 합성 데이터에 적합해보자. 데이터 생성 시, 다음 모델을 가정한다:

$$y = 0.5 + 2x - x^2 + \epsilon, \epsilon \sim \mathcal{N}(0, 1^2).$$

우리는 데이터를 생성하고, 다항식 기저를 사용하여, 최소자승해(least squares solution)를 통해 $\mathbf{w}$ 를 구할 것이다:

$$\mathbf{w} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y},$$

여기서 Φ 는 $N \times M$ 크기의 **디자인 행렬**(design matrix)로, n 번째 행은 $\phi(x_n)^T$ 이고, $\mathbf{y}$ 는 벡터 $(y_1, y_2, \dots, y_N)^T$ 이다.

1 선형 기저 함수 모델

● 선형 기저 함수 모델(Linear Basis Function Models)

```
[1]: import numpy as np

def polynomial_features(x, degree):
    """
    Given a 1D array x and an integer degree,
    return a design matrix Phi for polynomial basis functions:
    [1, x, x^2, ..., x^degree].
    """
    x = np.asarray(x)
    # np.vander creates a Vandermonde matrix
    # increasing=True : [1, x, x^2, ..., x^degree]
    Phi = np.vander(x, degree+1, increasing=True)
    return Phi
```

1

선형 기저 함수 모델

● 선형 기저 함수 모델(Linear Basis Function Models)

```
[2]: print(polynomial_features([2],2))  
      print(polynomial_features([1,2,3,4,5],2))
```

```
[[1 2 4]]  
[[ 1  1  1]  
 [ 1  2  4]  
 [ 1  3  9]  
 [ 1  4 16]  
 [ 1  5 25]]
```


1 선형 기저 함수 모델

● 선형 기저 함수 모델(Linear Basis Function Models)

```
[3]: # Generate synthetic data
np.random.seed(1)
N = 50
X = np.linspace(-2, 2, N)
true_w = np.array([0.5, 2.0, -1.0])
y_true = true_w[0] + true_w[1]*X + true_w[2]*(X**2)
noise = np.random.normal(0, 1, size=N)
y = y_true + noise
```

1 선형 기저 함수 모델

● 선형 기저 함수 모델(Linear Basis Function Models)

```
[4]: # Build polynomial design matrix (up to x^2)
Phi = polynomial_features(X, degree=2)

# Solve the normal equations
w_est = np.linalg.inv(Phi.T @ Phi) @ (Phi.T @ y)
print("Estimated parameters:", w_est)
```

```
Estimated parameters: [ 0.30818627  2.10168187 -0.88016698]
```

$$y = 0.5 + 2x - x^2 + \epsilon,$$

1 선형 기저 함수 모델

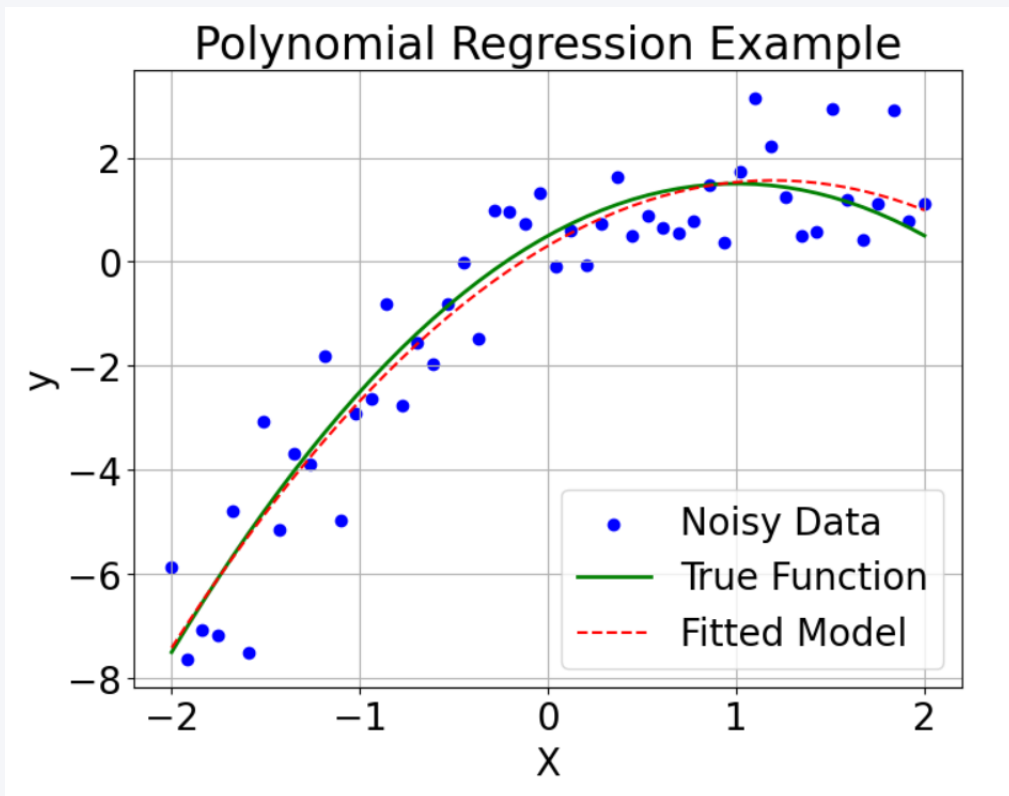
● 선형 기저 함수 모델(Linear Basis Function Models)

```
[8]: # Predict with the fitted model
y_pred = Phi @ w_est

import matplotlib.pyplot as plt
plt.rcParams.update({'font.size': 20}) # 전역 폰트 크기를 20로 설정
plt.figure(figsize=(8,6))
plt.scatter(X, y, label='Noisy Data', color='blue')
plt.plot(X, y_true, label='True Function', color='green', linewidth=2)
plt.plot(X, y_pred, label='Fitted Model', color='red', linestyle='--')
plt.legend()
plt.title("Polynomial Regression Example")
plt.xlabel("X")
plt.ylabel("y")
plt.grid(True)
plt.show()
```

1 선형 기저 함수 모델

● 선형 기저 함수 모델(Linear Basis Function Models)



1 선형 기저 함수 모델

● 선형 회귀 모델의 최대 우도 해석

$$y(x, \mathbf{w}) = \mathbf{w}^T \boldsymbol{\phi}(x) = \sum_{j=0}^{M-1} w_j \phi_j(x).$$

$$L(\mathbf{w}) = \sum_{n=1}^N (y_n - \mathbf{w}^T \boldsymbol{\phi}(x_n))^2.$$

$$\mathbf{w}_{\text{LS}} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}.$$

1 선형 기저 함수 모델

● 선형 회귀 모델의 최대 우도 해석

$$L(\mathbf{w}) = \sum_{n=1}^N (y_n - \mathbf{w}^T \phi(x_n))^2 = (\mathbf{y} - \Phi \mathbf{w})^T (\mathbf{y} - \Phi \mathbf{w}).$$

유도과정(벡터 미분):

$$\begin{aligned} \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} &= \frac{\partial}{\partial \mathbf{w}} \{ (\mathbf{y} - \Phi \mathbf{w})^T (\mathbf{y} - \Phi \mathbf{w}) \} \\ &= -2\Phi^T (\mathbf{y} - \Phi \mathbf{w}) = \mathbf{0}. \end{aligned}$$

위 식의 해를 $\mathbf{w}^*$ 로 표기하고 정리하면,

$$-\Phi^T \mathbf{y} + \Phi^T \Phi \mathbf{w}^* = \mathbf{0},$$

$$(\Phi^T \Phi) \mathbf{w}^* = \Phi^T \mathbf{y}.$$

$$\mathbf{w}^* = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}.$$

1

선형 기저 함수 모델

● 선형 회귀 모델의 최대 우도 해석

- 관측된 데이터 t_n 는 모델 $y(x_n, \mathbf{w})$ 에 다음의 독립적인 정규분포 잡음이 더해져 생성된다고 가정한다:

$$t_n = \mathbf{w}^T \phi(x_n) + \epsilon_n, \quad \text{where } \epsilon_n \sim \text{iid } \mathcal{N}(0, \sigma^2).$$

- 파라미터 $\mathbf{w}$ 와 입력 $X = (x_1, \dots, x_N)$ 이 주어졌을 때, 데이터 $\mathbf{t} = (t_1, \dots, t_N)$ 를 관찰할 확률(우도)은 다음과 같다:

$$p(\mathbf{t}|\mathbf{w}, X) = \prod_{n=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(t_n - \mathbf{w}^T \phi(x_n))^2}{2\sigma^2}\right).$$

- 이 우도의 로그를 취한 **로그우도**는 다음과 같이 표현된다:

$$\ln p(\mathbf{t}|\mathbf{w}, X) = -\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=1}^N (t_n - \mathbf{w}^T \phi(x_n))^2.$$

2

정규화된 최소자승법



2 정규화된 최소자승법

● 정규화된 최소자승법(Regularized Least-Squares)

훈련 데이터 $D = \{(x_n, t_n)\}_{n=1}^N$ 가 주어졌을 때, 최소자승법의 성능 지수(Performance Index, PI)를 다음과 같이 정의할 수 있다.

최소자승법의 성능 지수:

$$\varepsilon_{LS} = \frac{1}{2} \sum_{n=1}^N (t_n - \mathbf{w}^T \phi(x_n))^2.$$

정규화된 최소자승법의 성능 지수:

$$\varepsilon_{\text{Reg.LS}} = \varepsilon_{LS} + \lambda \varepsilon_w(\mathbf{w}).$$

2 정규화된 최소자승법

● 정규화된 최소자승법(Regularized Least-Squares)

L2 정규화 방법에서는 ε_w 를 다음과 같이 정의한다:

$$\varepsilon_w(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} = \frac{1}{2} \|\mathbf{w}\|_2^2.$$

따라서,

$$\varepsilon_{\text{Reg.LS}} = \frac{1}{2} \|\mathbf{t} - \Phi \mathbf{w}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2.$$

여기서, $\|\mathbf{t} - \Phi \mathbf{w}\|^2 = (\mathbf{t} - \Phi \mathbf{w})^T (\mathbf{t} - \Phi \mathbf{w}) = \sum_{n=1}^N (t_n - \mathbf{w}^T \phi(x_n))^2$ 이다.

2 정규화된 최소자승법

● 정규화된 최소자승법(Regularized Least-Squares)

$$\begin{aligned}\frac{\partial \varepsilon_{\text{Reg.LS}}}{\partial \mathbf{w}} &= \frac{\partial}{\partial \mathbf{w}} \left\{ \frac{1}{2} (\mathbf{t} - \Phi \mathbf{w})^T (\mathbf{t} - \Phi \mathbf{w}) + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w} \right\} \\ &= -\Phi^T (\mathbf{t} - \Phi \mathbf{w}) + \lambda \mathbf{w} = \mathbf{0}.\end{aligned}$$

$$-\Phi^T \mathbf{t} + \Phi^T \Phi \mathbf{w}^* + \lambda \mathbf{w}^* = \mathbf{0},$$

$$(\Phi^T \Phi + \lambda I) \mathbf{w}^* = \Phi^T \mathbf{t}.$$

$$\mathbf{w}^* = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T \mathbf{t}.$$

3

베이지안 선형 회귀



3

베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)

사전분포 (Prior):

$$\boldsymbol{w} \sim \mathcal{N}(\boldsymbol{m}_0, S_0)$$

iID training data:

$$D = \{(x_n, t_n)\}_{n=1}^N$$

우도 (Likelihood):

$$P(D | \boldsymbol{w})$$

사후분포 (Posterior):

$$P(\boldsymbol{w} | D) = \frac{P(\boldsymbol{w}, D)}{P(D)} \propto P(D | \boldsymbol{w})P(\boldsymbol{w})$$

3

베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)

Model:

$$t = \mathbf{w}^T \phi(x) + \epsilon, \quad \text{where } \epsilon \sim \text{iid } \mathcal{N}(0, \beta^{-1}).$$

여기서, β 는 precision으로 분산의 역수를 의미하고, 우선은 알려진 상수 값이라고 가정한다.

3 베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)

IID Training Data:

$$D = \{(x_n, t_n)\}_{n=1}^N$$

행렬-벡터 표현:

$$\begin{bmatrix} t_1 \\ \vdots \\ t_N \end{bmatrix} = \begin{bmatrix} \phi^T(x_1) \\ \vdots \\ \phi^T(x_N) \end{bmatrix} \mathbf{w} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_N \end{bmatrix},$$

또는 간단히

$$\mathbf{t} = \Phi \mathbf{w} + \boldsymbol{\epsilon}.$$

3

베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)

사전분포 (Prior):

$$\mathbf{w} \sim \mathcal{N}(\mathbf{m}_0, S_0),$$

여기서, $\mathbf{m}_0 = \mathbf{0}$, $S_0 = \alpha^{-1}I$ 로 가정하면 (α 는 precision),

$$P(\mathbf{w}) \propto \exp\left(-\frac{\alpha}{2}\mathbf{w}^T\mathbf{w}\right).$$

우도 (Likelihood):

$$t \sim \mathcal{N}(\Phi\mathbf{w}, \beta^{-1}I)$$

$$P(D|\mathbf{w}) \propto \exp\left(-\frac{\beta}{2}(\mathbf{t} - \Phi\mathbf{w})^T(\mathbf{t} - \Phi\mathbf{w})\right)$$

3 베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)

사후분포 (Posterior):

$$P(\mathbf{w}|D) = \frac{P(\mathbf{w}, D)}{P(D)} \propto P(D|\mathbf{w})P(\mathbf{w})$$

사후분포는 우도와 사전분포의 곱에 비례하므로,

$$P(\mathbf{w}|D) \propto \exp\left(-\frac{\beta}{2}(\mathbf{t} - \Phi\mathbf{w})^T(\mathbf{t} - \Phi\mathbf{w})\right) \exp\left(-\frac{\alpha}{2}\mathbf{w}^T\mathbf{w}\right).$$

$$P(\mathbf{w}|D) : \mathbf{w} \sim \mathcal{N}(\mathbf{m}_N, S_N),$$

$$\mathbf{m}_N = \left(\Phi^T\Phi + \frac{\alpha}{\beta}I\right)^{-1} \Phi^T\mathbf{t}, \quad S_N = (\beta\Phi^T\Phi + \alpha I)^{-1}.$$

3 베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)

예측분포:

예측분포는 새로운 입력 x^* 가 주어졌을 때 사후분포를 이용하여 출력 t^* 를 예측하는 분포로 다음과 같다:

$$P(t^*|x^*, D) = \int P(t^*|x^*, \mathbf{w})P(\mathbf{w}|D)d\mathbf{w}.$$

여기서,

$$P(t^*|x^*, \mathbf{w}) = \mathcal{N}(t^*|\phi(x^*)^T \mathbf{w}, \beta^{-1}).$$

● 베이지안 선형 회귀(Bayesian linear Regression)

참고: 다변량 정규분포의 성질 (Bishop, p93) ¶

다변량 정규분포 $\mathbf{x}$ 와 조건부분포 $\mathbf{y}|\mathbf{x}$ 가 다음과 같이 주어질 때,

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \Lambda^{-1})$$

$$p(\mathbf{y}|\mathbf{x}) = \mathcal{N}(\mathbf{y}|A\mathbf{x} + \mathbf{b}, L^{-1})$$

$\mathbf{y}$ 의 주변분포와 조건부분포 $\mathbf{x}|\mathbf{y}$ 는 다음과 같다:

$$p(\mathbf{y}) = \mathcal{N}(\mathbf{y}|A\boldsymbol{\mu} + \mathbf{b}, L^{-1} + A\Lambda^{-1}A^T)$$

$$p(\mathbf{x}|\mathbf{y}) = \mathcal{N}(\mathbf{x}|\Sigma\{A^T L(\mathbf{y} - \mathbf{b}) + \Lambda\boldsymbol{\mu}\}, \Sigma)$$

여기서,

$$\Sigma = (\Lambda + A^T L A)^{-1}.$$

3 베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)

다음의 정규분포들이 주어질 때,

$$P(\mathbf{w}|D) = \mathcal{N}(\mathbf{w} | \mathbf{m}_N, S_N)$$
$$P(t^*|x^*, \mathbf{w}) = \mathcal{N}(t^* | \phi(x^*)^T \mathbf{w}, \beta^{-1})$$

t^* 의 주변분포는 다음과 같다:

$$P(t^*|x^*, D) = \mathcal{N}(t^* | \phi(x^*)^T \mathbf{m}_N, \beta^{-1} + \phi(x^*)^T S_N \phi(x^*)),$$
$$= \mathcal{N}(t^* | \mathbf{m}_N^T \phi(x^*), \sigma_N^2(x^*)).$$

3 베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)

```
# t = y(x,w) + epsilon
t = y_true + noise

# 베이지안 선형 회귀 하이퍼파라미터 설정
alpha = 1.0 # 사전분포의 정밀도
beta = 1.0 # 관측 노이즈의 정밀도

# 사전분포 (prior dist): mean = 0 벡터, precision = alpha*I
prior_mean = np.zeros(Phi.shape[1])
prior_precision = alpha * np.eye(Phi.shape[1])
```

3 베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)

```
# Likelihood: precision = beta
# 사후분포 (posterior dist): combine prior and likelihood
posterior_precision = prior_precision + beta * Phi.T @ Phi
posterior_covariance = np.linalg.inv(posterior_precision)
posterior_mean = posterior_covariance @ (beta * Phi.T @ t)

# 예측분포 (predictive dist) 계산
def predict(Phi_new, posterior_mean, posterior_covariance, beta):
    """
    Compute the predictive mean and variance for new data using
    the posterior distribution.
    """
    mean = Phi_new @ posterior_mean
    variance = 1 / beta + np.diag(Phi_new @ posterior_covariance @ Phi_new.T)
    return mean, variance
```

출처:파이썬 프로그램 실습내용 중

3 베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)

```
# New data points for prediction
X_new = np.linspace(-2, 2, 100)
Phi_new = polynomial_features(X_new, degree=2)

# Predictive distribution
predictive_mean, predictive_variance = predict(Phi_new, posterior_mean, posterior_covariance, beta)
```

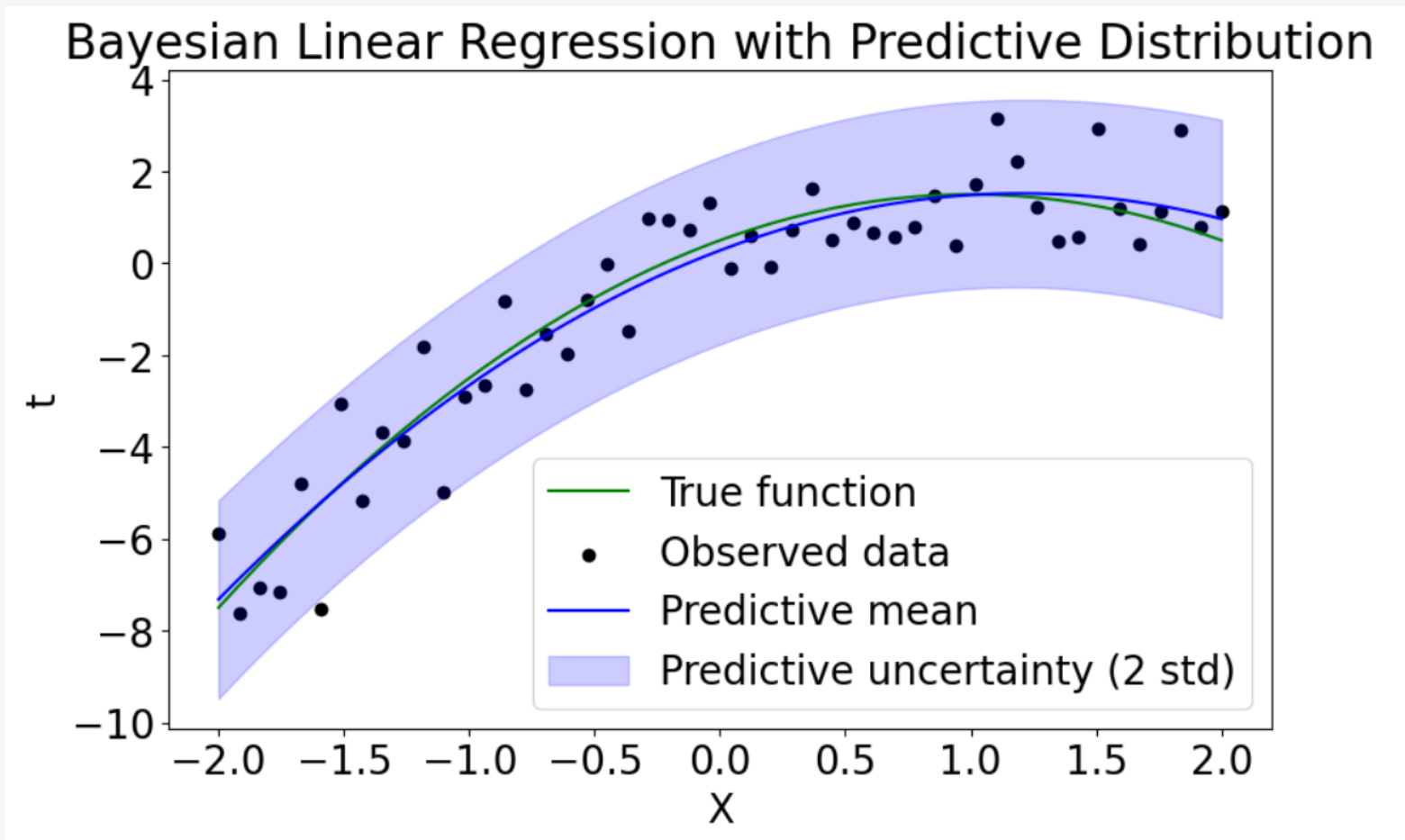
3 베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)

```
# Plot true curve, observed data, and predictive distribution
plt.figure(figsize=(10, 6))
plt.plot(X, y_true, label="True function", color="green")
plt.scatter(X, t, label="Observed data", color="black")
plt.plot(X_new, predictive_mean, label="Predictive mean", color="blue")
plt.fill_between(
    X_new,
    predictive_mean - 2 * np.sqrt(predictive_variance),
    predictive_mean + 2 * np.sqrt(predictive_variance),
    color="blue",
    alpha=0.2,
    label="Predictive uncertainty (2 std)"
)
plt.legend()
plt.xlabel("X")
plt.ylabel("t")
plt.title("Bayesian Linear Regression with Predictive Distribution")
plt.show()
```


3 베이지안 선형 회귀

● 베이지안 선형 회귀(Bayesian linear Regression)



출처:파이썬 프로그램 실습내용 중

✧ 정리하기

- 선형기저함수 모델

입력변수에 적용된 변환들의 선형 결합으로 출력변수 표현
일반최소자승법: 잔차제곱합 최소화, 최우추정법과 동일

- 정규화된 최소자승법

람다를 통한 파라미터 조절

- 베이즈안 선형 회귀

모델의 불확실성 정량화 및 예측구간 제시 가능

09 강

다음시간안내

선형 판별 분석

